

Mercury 跨平台复刻项目 — 第一次汇报

汇报时间：2026年7月13日 项目状态：Planning 汇报人：XXX 小组

一、项目目标

1.1 目标定位

Mercury — 一款跨平台 RSS 阅读器，将 macOS 原生应用复刻到 Windows、macOS、Linux 三大平台，提供舒适的信息聚合与阅读体验。

1.2 核心价值

价值点	描述
本地优先	无需账号注册/登录，所有数据存储在本地 SQLite 数据库，隐私保护优先
平台中立	一套代码支持 Windows 10+、macOS 12+、Linux (Wayland + X11)
大模型中立	通过 OpenAI 兼容协议接入任意云端/本地大模型，不绑定特定服务商
AI 赋能阅读	文章摘要、双语翻译、AI 标签建议，提升阅读效率

1.3 v1 必须实现的功能

- Feed 订阅与内容呈现：RSS/Atom/JSON Feed 订阅、OPML 导入导出、自动/手动同步
- 文章清洗与阅读体验：Readability 内容提取、Markdown 渲染、主题切换
- AI 摘要：文章摘要生成、流式输出、可配置语言与详细程度
- AI 双语翻译：段落级双语对照、多段落并发翻译、重试失败段落
- 标签系统：手动标签添加与筛选、AI 标签建议

二、人员分工

2.1 团队组成

成员	角色	核心职责
张三	后端核心开发	数据库层、Feed 解析与同步、文章提取算法、IPC API
李四	前端核心开发	UI 组件、状态管理、AI 交互界面、阅读体验优化
王五	全栈集成与 DevOps	AI 智能体实现、跨平台打包、CI/CD、测试与质量保障

2.2 职责划分

后端核心开发（张三）：

数据库层：SQLite Schema 设计、数据模型定义、迁移系统

Feed 解析: feed-rs 集成、后台同步任务调度、并发控制
文章提取: 基于 readability crate 实现内容提取、HTML 清洗、Markdown 转换
IPC API: Feed/文章/设置管理 API、错误处理与日志

前端核心开发（李四）：

UI 组件: 侧边栏、文章列表、阅读器、设置面板
状态管理: React Context + useReducer 全局状态、自定义 Hooks
AI 交互: 摘要面板、双语翻译对照布局、流式输出实时渲染
阅读体验: 主题切换、响应式设计、文章样式定制

全栈集成与 DevOps（王五）：

AI 智能体: OpenAI 兼容客户端、摘要/翻译/标签智能体
跨平台打包: Tauri 配置、各平台安装包构建
CI/CD: GitHub Actions 工作流、自动化测试与发布
测试保障: 单元测试、集成测试、E2E 测试

三、初步技术判断与决策

3.1 技术栈选型

层级	选型	理由
语言	Rust	跨平台一等公民、编译器即 reviewer、AI 训练数据丰富、Tauri 同语言单进程
UI 壳	Tauri 2	同语言零 IPC、WebView2/WKWebView/WebViewGTK 原生支持
前端	React + TypeScript	AI 生成代码质量最高，生态成熟
数据库	SQLite (rusqlite)	schema 参考原始 Mercury，WAL 模式支持并发读
HTTP	reqwest	Rust 生态标准客户端
Feed 解析	feed-rs	支持 RSS/Atom/JSON Feed 全格式
文章提取	readability crate	Mozilla Readability 的 Rust 移植，效果可靠
HTML 清洗	scraper	CSS 选择器友好，AI 辅助开发效率高
Markdown	comrak	GFM 完整支持（表格/任务列表/代码块）
AI 协议	OpenAI 兼容	覆盖 DeepSeek/Ollama/vLLM 等所有兼容服务

3.2 架构原则

- 业务逻辑不得依赖 UI — 核心逻辑独立于展示层
- 数据库层不得依赖平台 API — 确保跨平台兼容性
- 平台能力必须经过抽象层 — MercuryCore / MercuryStore 中不得出现平台特定 import
- UI 仅负责展示 — 所有业务逻辑走 Rust 核心
- 数据库单进程写入 — 避免并发写入冲突

3.3 关键技术决策

问题	方案	理由
Readability 文章提取	基于 readability crate	Mozilla 算法移植，效果可靠
AI 流式输出 IPC	Tauri Event API	原生支持，简单易用
SQLite Schema	参考原始 Mercury	保持数据模型一致，便于迁移
后台 Feed 同步	Rust Tokio 异步任务	高性能、内存安全
双语翻译布局	CSS Grid/Flexbox	灵活、响应式
跨平台路径	Rust dirs crate	标准路径，简单可靠

四、实现计划

4.1 阶段规划

阶段	时间	核心任务	交付物	负责人
阶段 1: 项目初始化	Week 1	Tauri + React 项目框架搭建、代码规范制定	项目基础框架、Cargo.toml、package.json	王五
阶段 2: 数据库与 API	Week 2	SQLite Schema 设计、数据模型、IPC API 定义	数据库层代码、API 接口文档	张三
阶段 3: Feed 订阅与同步	Week 3	Feed 解析集成、后台同步任务、订阅源管理	Feed 模块代码、同步逻辑	张三
阶段 4: 文章阅读体验	Week 4	Readability 提取、Markdown 渲染、阅读器 UI	文章提取代码、阅读器组件	张三 + 李四
阶段 5: AI 智能体	Week 5	AI 客户端、摘要/翻译/标签智能体、前端交互	AI 模块代码、AI 交互组件	王五 + 李四
阶段 6: 跨平台打包	Week 6	Tauri 配置、各平台安装包、CI/CD 流水线	安装包、GitHub Actions 配置	王五
阶段 7: 测试与发布	Week 7	单元测试、集成测试、Bug 修复、文档完善	测试报告、发布版本	三人协作

4.2 当前进展

- 已完成 INIT.md（项目目标、功能规划、技术约束）
- 已完成 INIT(2).md（详细功能分解、技术选型、架构原则）
- 已完成 AGENTS.md（关键技术决策与约束固化）
- 下一步：生成 PLAN.md（分步执行计划）

4.3 风险与应对

风险	影响	应对措施
Feed 格式兼容性	部分订阅源解析失败	使用 feed-rs 成熟库，添加错误处理
Readability 提取质量	某些网站内容提取不准确	参考 Mozilla 算法，提供手动修正入口

跨平台路径处理	不同平台路径差异	使用 dirs crate 抽象，避免硬编码
Linux WebViewGTK	依赖安装复杂	提供详细安装指南，CI 自动化测试
LLM 接口稳定性	流式输出中断	实现重试机制，缓存已接收内容

五、总结

本次汇报完成了 Mercury 跨平台复刻项目的前期规划工作：

- 1. 明确了项目目标 — 跨平台、本地优先、AI 赋能的 RSS 阅读器
- 2. 确定了技术选型 — Rust + Tauri + React + SQLite 技术栈
- 3. 制定了人员分工 — 后端核心、前端核心、全栈集成三角色
- 4. 规划了实现路线 — 7 个阶段，7 周完成 v1 版本

下一步计划：生成 PLAN.md，启动 Phase 1 项目初始化工作。